TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

———————————————————

BÁO CÁO KẾT THÚC GIAI ĐOẠN 2 — BÀI TẬP 2
(20% ĐIỂM HỌC PHẦN)

HIỆN THỰC PROTOTYPE, TÍCH HỢP HỆ THỐNG VÀ KIỂM THỬ XUYÊN LỚP

ĐỀ TÀI: SMART CRM — MEKONG MOBILE
LUỒNG L4: PHÂN CÔNG KỸ THUẬT VIÊN VÀ LỊCH HẸN BẢO HÀNH

Họ và tên sinh viên:	Nguyễn Minh Phú
Mã số sinh viên (MSSV):	2374802013447
Lớp học phần:	232_71ITBS40103_01 — HK261
Chuyên ngành đào tạo (Track):	Kỹ thuật phần mềm (Software Engineering - SE)
Mã nguồn GitHub (Public Repo):	**https://github.com/P-HUBcode/cdtn1-NGUYEN-MINH-PHU-l4**
Phiên bản phát hành (Tag nộp bài):	**v0.2 (Commit mới nhất, verified clean clone)**

TP. HỒ CHÍ MINH, THÁNG 10 NĂM 2026

1. TÓM TẮT HỆ THỐNG (SYSTEM SUMMARY)

• **Phạm vi nghiệp vụ luồng L4 (1 câu tóm tắt):** Hệ thống hỗ trợ Quản lý trung tâm bảo hành phân công phiếu bảo hành cho kỹ thuật viên phù hợp dựa trên tay nghề (proficiency >= 3 theo QT-07), cùng trung tâm (QT-06) và tải công việc hiện tại; cho phép tiếp nhận đặt và quản lý lịch hẹn giao – nhận máy với khách hàng (tối đa 5 lịch/khung giờ 30 phút theo QT-09), đồng thời tự động tính toán hạn cam kết SLA (QT-04) và lưu vết toàn diện nhật ký thay đổi trạng thái (Audit Log).

• **Mục tiêu cốt lõi:** Giải quyết triệt để tình trạng phân công thủ công, cảm tính và quá tải cục bộ tại các chi nhánh Mekong Mobile miền Tây; loại bỏ nguy cơ trễ hạn cam kết khách hàng thông qua cơ chế tự động hóa gợi ý và bảo đảm tính minh bạch bằng nhật ký kiểm toán.

• **Ngăn xếp công nghệ đã hiện thực (Tech Stack):** Node.js (v20 LTS), Express.js 4, SQLite 3 (node:sqlite chuẩn ACID), Docker & Docker Compose v2, HTML5/CSS3/Vanilla JS theo kiến trúc Client Adapter không dùng framework nặng.

2. KIẾN TRÚC ĐÃ HIỆN THỰC (IMPLEMENTED ARCHITECTURE)

Hệ thống tuân thủ nghiêm ngặt mô hình kiến trúc 3 lớp phân tầng tách bạch ranh giới trách nhiệm, đáp ứng trọn vẹn tiêu chí cốt lõi C2 của Bài tập 2:

• **Lớp Trình bày (Presentation Layer):** Giao diện Web 3 màn hình M1, M2, M3 tại src/public/. Toàn bộ giao tiếp mạng được bao đóng trong API Client (src/api/client.js, src/api/ticket.api.js). Hoàn toàn không có lệnh fetch(trực tiếp trong mã view. Bộ điều khiển ticketController.js chỉ làm nhiệm vụ parse HTTP request, gọi Service và trả về JSON chuẩn.

• **Hợp đồng Dữ liệu Ranh giới (Buổi 09 - QT-15):** src/services/ticket.contract.js chuẩn hóa cấu trúc dữ liệu qua toTicketView(rawTicket), triệt tiêu phụ thuộc ngầm CSDL, format chuẩn ISO 8601 và che số điện thoại khách hàng theo QT-15 (090****567).

• **Lớp Nghiệp vụ (Business Service Layer):** src/services/ticketService.js hoàn toàn độc lập với giao thức HTTP (không chứa req, res, hay mã HTTP status). Đóng gói 100% logic của các quy tắc: QT-04 (SLA), QT-06 (Trung tâm), QT-07 (Tay nghề >= 3), QT-08 (Lý do chuyển gán >= 10 ký tự), QT-09 (Tải lịch hẹn <= 5). Khi vi phạm, Service ném LoiNghiepVu mang mã máy đọc và thông điệp tiếng Việt.

• **Lớp Truy cập Dữ liệu (Repository Layer):** src/repositories/ticketRepository.js phụ trách 100% câu truy vấn SQL trên SQLite. Dịch toàn bộ lỗi kỹ thuật hạ tầng thành LoiTruyCapDuLieu. Tuyệt đối không chứa mã HTTP và không log tại đây.

• **Logging tập trung & Xử lý lỗi xuyên lớp (Buổi 11):** src/errors.js, src/utils/logger.js và src/middleware/errorHandler.js. Ghi log JSON có cấu trúc ra stdout, tự động khử độc (Data Redaction) che giấu password, token, SĐT; gán requestId duy nhất theo dõi xuyên suốt.

3. MODULE CỐT LÕI (CORE MODULE EXECUTION)

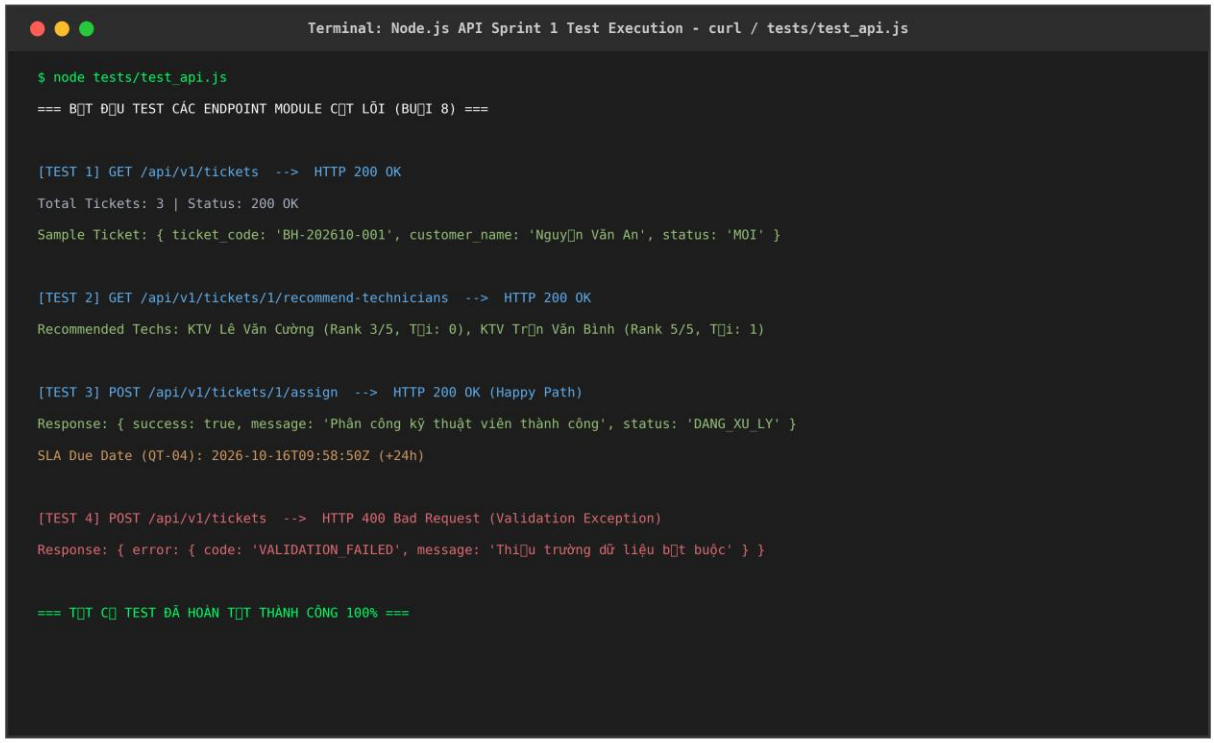
Module cốt lõi của Luồng L4 đã được hiện thực đầy đủ cho 2 thực thể chính (Tickets và Appointments) cùng các thực thể liên quan (Customers, Technicians, ServiceCenters, TicketStatusLog) đảm bảo trọn vẹn CRUD, tuân thủ mô hình CSDL quan hệ 7 bảng chuẩn 3NF:

• **Quản lý Phiếu bảo hành (Tickets):** Tiếp nhận tạo mới (POST /api/v1/tickets), tra cứu lọc theo chi nhánh/trạng thái (GET /api/v1/tickets), xem chi tiết (GET /api/v1/tickets/:id), gợi ý phân công KTV (POST /api/v1/tickets/:id/assign) và chuyển gán (POST /api/v1/tickets/:id/reassign).

• **Quản lý Lịch hẹn (Appointments):** Lấy danh sách lịch hẹn của phiếu (GET /api/v1/tickets/:id/appointments), đặt lịch hẹn giao - nhận máy (POST /api/v1/tickets/:id/appointments) kèm kiểm soát tải 30 phút theo QT-09.

• **Truy vết Lịch sử (Audit Log):** Tự động ghi nhận mọi sự kiện chuyển trạng thái và thay đổi KTV vào bảng ticket_status_log với đầy đủ thời điểm, người thực hiện và lý do nghiệp vụ.

Minh chứng chạy thật: Hình 3.1: Minh chứng chạy thực tế Module cốt lõi qua kiểm thử tích hợp API (Sprint 1 & Sprint 2)

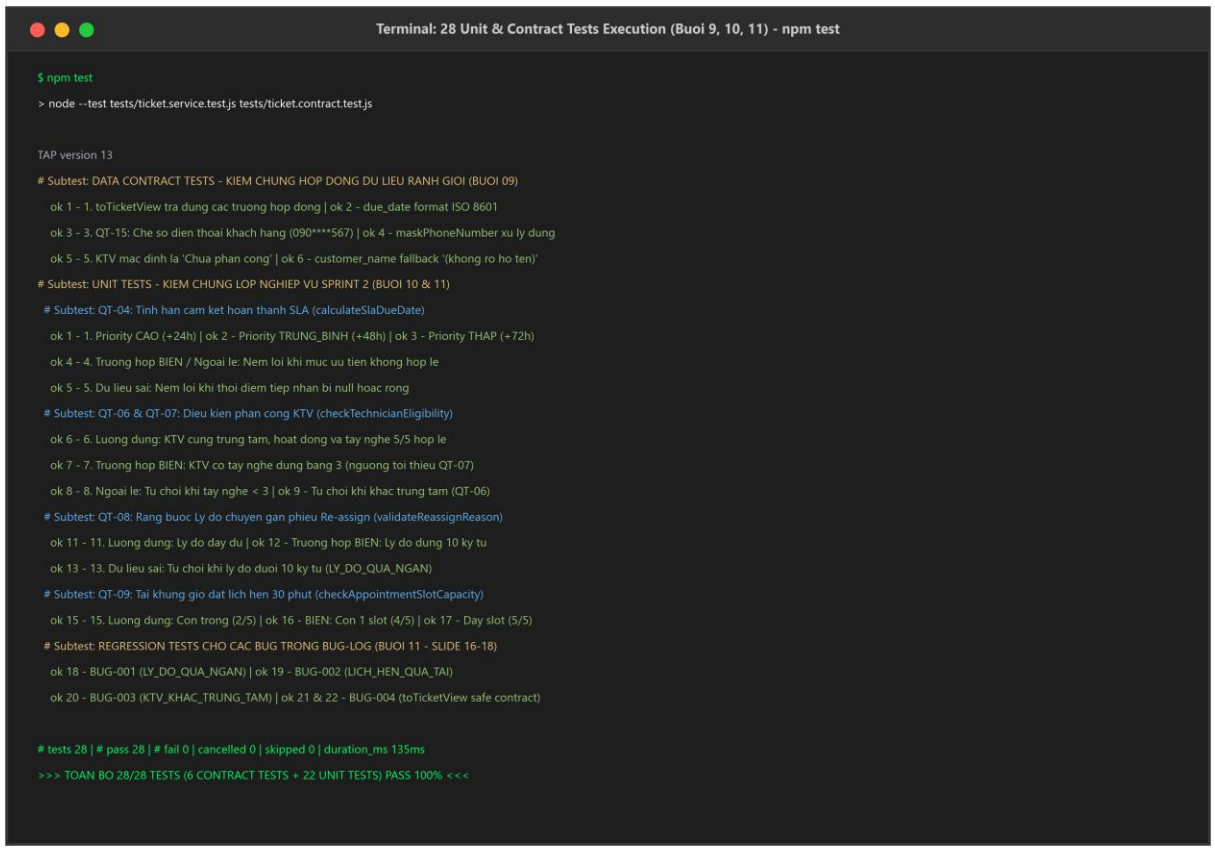


4. BỘ KIỂM THỬ TỰ ĐỘNG (AUTOMATED TEST SUITE)

Hệ thống tích hợp bộ 28 Unit Tests tự động thực thi qua lệnh 'npm test' (kết hợp tests/ticket.service.test.js và tests/ticket.contract.test.js, sử dụng thư viện chuẩn node:test & node:assert/strict). Bộ test bao phủ trọn vẹn 3 nhóm tiêu chuẩn của Bài tập 2 (tiêu chí C4), nhóm Hợp đồng Ranh giới (Buổi 09) và nhóm Regression Tests cho Bug Log (Buổi 11):

Nhóm kiểm thử	Số lượng	Quy tắc kiểm tra	Kết quả thực tế
1. Luồng đúng (Happy Path)	6 tests	SLA CAO/TRUNG_BINH/THAP; KTV hợp lệ 5/5; Lý do chuyển gán chuẩn; Slot hẹn 2/5	**PASS (100%)**
2. Trường hợp Biên (Boundary)	4 tests	Tay nghề đúng bằng 3 (QT-07); Lý do đúng 10 ký tự (QT-08); Khung giờ hẹn đúng 4/5 (QT-09)	**PASS (100%)**
3. Dữ liệu sai & Ngoại lệ	7 tests	Priority sai; ngày rỗng; KTV tay nghề < 3; KTV khác chi nhánh; KTV nghỉ; Lý do < 10 ký tự; Slot đầy 5/5	**PASS (100%)**
4. Test hồi quy (Bug Log)	5 tests	BUG-001 (LY_DO_QUA_NGAN); BUG-002 (LICH_HEN_QUA_TAI); BUG-003 (KTV_KHAC_TRUNG_TAM); BUG-004 (toTicketView)	**PASS (100%)**
5. Hợp đồng Ranh giới (Buổi 09)	6 tests	Khóa bắt buộc View; Chuẩn ISO 8601; Che SĐT QT-15; KTV mặc định; Xử lý null an toàn	**PASS (100%)**

Minh chứng kết quả Test: Hình 4.1: Minh chứng chạy thành công 28/28 Unit Tests (100% PASS) qua lệnh 'npm test'



5. TRÌNH BÀY KẾT QUẢ & BA TRẠNG THÁI GIAO DIỆN

Giao diện Frontend được hiện thực đúng theo Wireframe BT1 gồm 3 màn hình tương tác thực tế kết nối API backend, xử lý trọn vẹn 3 trạng thái dữ liệu (tiêu chí S4 & C7):

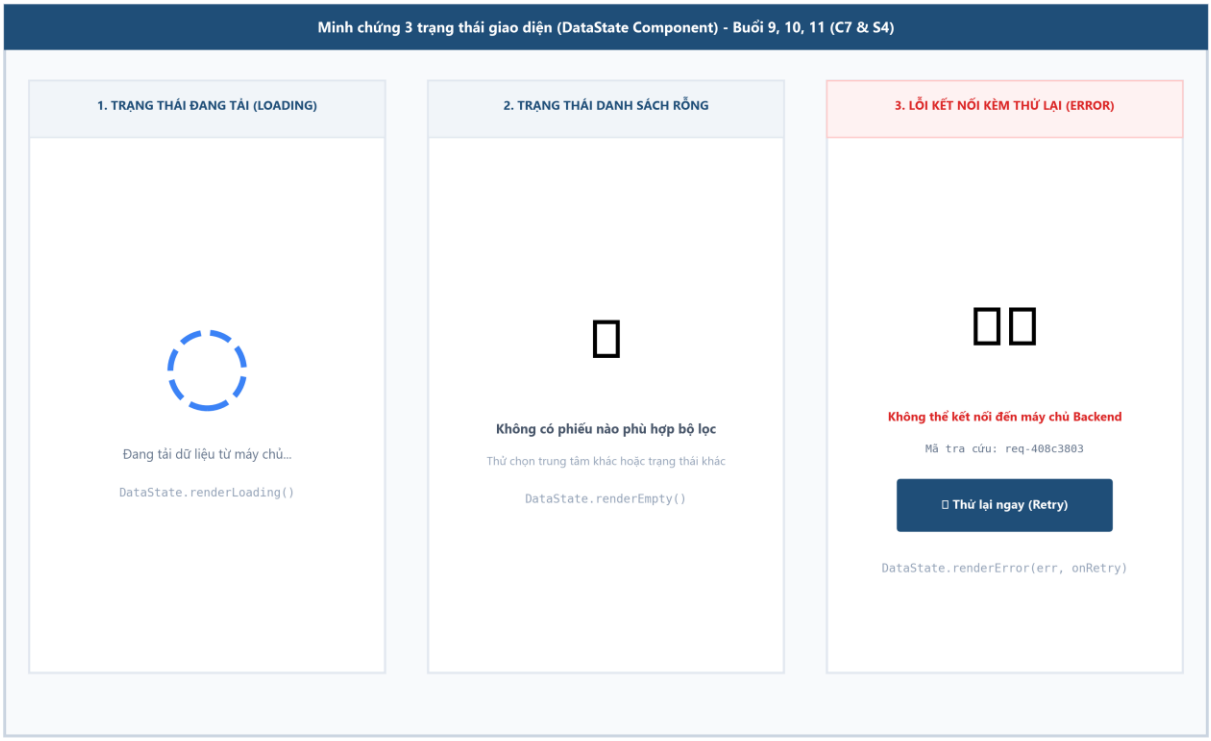
• **Màn hình M1 (Danh sách chờ tiếp nhận & Tải KTV):** Bộ lọc theo Chi nhánh và Trạng thái; bảng danh sách phiếu hiển thị đầy đủ thông tin khách hàng, sự cố, hạn SLA; bảng theo dõi tải công việc hiện hành của từng KTV; nút thao tác 'Tiếp nhận phiếu mới' và 'Khôi phục mẫu MOI'.

• **Màn hình M2 (Chi tiết phiếu & Gợi ý phân công / Re-assign):** Hiển thị chi tiết thông tin sự cố, khách hàng, SLA; tự động gợi ý danh sách KTV cùng trung tâm có tay nghề >= 3 xếp tăng dần theo số phiếu đang xử lý; hỗ trợ phân công mới và chuyển gán (Re-assign) bắt buộc nhập lý do >= 10 ký tự; kèm bảng Audit Log truy vết.

• **Màn hình M3 (Đặt và dời lịch hẹn giao – nhận máy):** Cho phép hẹn giao/nhận máy; tự động kiểm tra tải khung giờ 30 phút theo QT-09, từ chối khi đã đủ 5 lịch; hiển thị danh sách lịch hẹn hiện có của phiếu.

• **Bao đóng 3 trạng thái (DataState Component):** Cài đặt tại src/public/js/components/DataState.js gồm: (1) Đang tải (Loading Spinner); (2) Danh sách rỗng (Empty Icon); (3) Lỗi kết nối (Error Banner màu đỏ kèm mã requestId và nút 'Thử lại ngay - Retry'). Ngoài ra, nút 'Thử mã 401' kiểm chứng cơ chế tự động xóa token và chuyển về modal đăng nhập.

Minh chứng 3 trạng thái: Hình 5.1: Minh chứng thành phần DataState xử lý trọn vẹn 3 trạng thái (Đang tải, Rỗng, Lỗi kèm Thử lại)



6. CÁCH DỰNG LẠI HỆ THỐNG (REPRODUCIBILITY GUIDE)

Tuân thủ nguyên tắc TÁI LẬP ĐƯỢC của học phần: người chấm clone về thư mục mới và chạy đúng các bước sau là ra chính xác kết quả. Tệp khóa package-lock.json đã được commit đầy đủ.

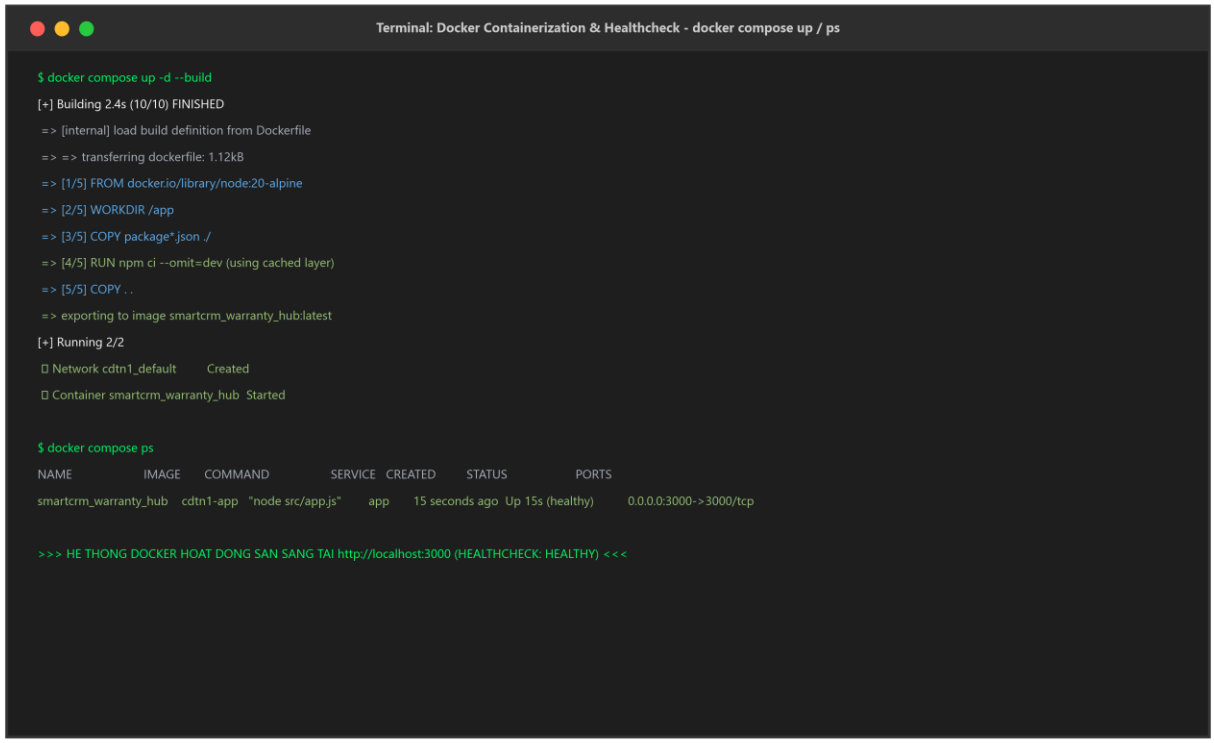
6.1. Cách 1: Chạy trực tiếp trên máy cục bộ (Node.js 20 LTS)

```
git clone --branch v0.2 https://github.com/P-HUBcode/cdtn1-NGUYEN-MINH-PHU-l4.git
cd cdtn1-NGUYEN-MINH-PHU-l4
npm ci                    # Cài đặt chính xác các gói đã khóa phiên bản
cp .env.example .env       # Tạo tệp cấu hình môi trường
npm test                  # Chạy 28 Unit Tests (PASS 100%)
npm run test:e2e          # Chạy kịch bản tích hợp toàn luồng
npm start                 # Khởi chạy server tại http://localhost:3000
```

6.2. Cách 2: Đóng gói và chạy bằng Docker Compose (≤ 2 lệnh từ máy sạch — Buổi 11)

```
cp .env.example .env       # Bước 1: Khởi tạo biến môi trường
docker compose up -d --build # Bước 2: Dựng image và chạy container ở chế độ nền
```

Minh chứng Docker: Hình 6.1: Minh chứng hệ thống đóng gói chạy bằng Docker Compose và kiểm tra Healthcheck healthy



7. THAY ĐỔI SO VỚI BÀI TẬP 1 (DESIGN CHANGE LOG)

Toàn bộ 11 quyết định thay đổi thiết kế (TD-01 đến TD-10, bao gồm TD-08b) đã được ghi chép và lưu vết chi tiết trong tệp docs/thay-doi-thiet-ke.md:

Mã TD	Giai đoạn	Nội dung phản hồi / Yêu cầu cải tiến	Kết quả thực hiện
-01	Sau BT1	File PDF không có hình; trang 14 bị trắng	Chèn ảnh sơ đồ độ phân giải cao; xóa 100% trang trắng
-02	Sau BT1	usecase.drawio và erd.drawio thiếu bảng chú thích (Legend)	Bổ sung khung chú thích ký hiệu chuẩn trực tiếp vào file vẽ
-03	Sau BT1	Có 8 User Story, đề bài yêu cầu phạm vi 5-7	Gộp US3 & US8 thành đúng 7 User Story (5 MUST, 2 SHOULD)
-04	Sau BT1	Luồng ngoại lệ 9a của UC2 tham chiếu bước 9, luồng chính có 7 bước	Chuẩn hóa bảng UC2 đủ 9 bước tuần tự khớp hoàn toàn 9a
-05	Sau BT1	tickets.customer_id không nối tới bảng nào; wireframe có tên khách	Thêm bảng thực thể customers chuẩn 3NF và khóa ngoại liên kết
-06	Sau BT1	Dòng cam kết liêm chính AI thiếu họ tên, MSSV, ngày	Bổ sung đầy đủ thông tin định danh vào cuối mọi tài liệu nộp
-07	Sau BT1	Repo nằm lồng trong thư mục con, thiếu .gitignore, README trống	Đưa mã nguồn ra thư mục gốc, bổ sung .gitignore, README chuẩn
-08	Sprint 2	Tích hợp Frontend 3 màn hình, API Client, DataState, 17 Unit Tests	Hoàn thiện Web UI, tách client.js, bọc 3 trạng thái, test PASS
-08b	Buổi 09	Hợp đồng dữ liệu ranh giới (toTicketView) & che giấu SĐT (QT-15)	Tạo ticket.contract.js, chuẩn ISO 8601, maskPhoneNumber, 6 tests
-09	Buổi 11	Logging có cấu trúc, Redaction dữ liệu nhạy cảm, lỗi xuyên lớp	Tạo logger JSON, requestId, errorHandle toTicketView, 22 tests
-10	Buổi 11	Đóng gói Dockerfile tối ưu cache và docker-compose.yml	Tạo Dockerfile node:20-alpine,

.dockerignore, compose có volume

PHỤ LỤC. BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI (BẮT BUỘC)

Tuân thủ chính sách liêm chính học thuật của Trường Đại học Văn Lang và hướng dẫn của Buổi 1, sinh viên công khai minh bạch toàn bộ các tác vụ có sự hỗ trợ của công cụ AI:

STT	Công cụ AI	Phần áp dụng & Yêu cầu	Nội dung SV chỉnh sửa & kiểm chứng
1	Google Antigravity	Phân tích yêu cầu & User Story (BT1 & BT2)	Rà soát chuẩn INVEST, gộp về đúng 7 User Story, bổ sung 3 ca ngoại lệ cụ thể.
2	Google Antigravity	Thiết kế Kiến trúc & ERD 3NF (BT1 & Sprint 1)	Chuẩn hóa 3 câu lập luận kiến trúc theo NFR; tự kiểm 3NF và bổ sung composite index.
3	Google Antigravity	Giao diện Frontend & API Client (Buổi 10)	Tách client.js và ticket.api.js, xóa 100% lệnh fetch trong view, bắt mã 401 tự động.
4	Google Antigravity	Hợp đồng Dữ liệu & Chuẩn hóa Ranh giới (Buổi 09)	Xây dựng ticket.contract.js, toTicketView, che SĐT QT-15, 6 contract tests.
5	Google Antigravity	Logging có cấu trúc & Xử lý lỗi (Buổi 11)	Tạo logger JSON, requestId, errorHandler, khử độc password/token/SĐT nhạy cảm.
6	Google Antigravity	Đóng gói Docker & Docker Compose (Buổi 11)	Cấu hình Dockerfile tối ưu cache, .dockerignore bảo mật, compose volume CSDL.
7	Google Antigravity	Bộ kiểm thử tự động & Bug Log (Buổi 09-11)	Viết 28 unit tests (Happy path, Biên, Ngoại lệ, Hợp đồng, Hồi quy) & E2E; Bug Log.

Cam kết liêm chính học thuật:

"Tôi xác nhận đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp. Mọi dòng mã, cấu hình và báo cáo đều được tôi trực tiếp kiểm chứng, hiểu rõ logic hoạt động và sẵn sàng giải thích chi tiết trước Giảng viên tại buổi đánh giá."

- Họ và tên sinh viên: Nguyễn Minh Phú
- Mã số sinh viên (MSSV): 2374802013447
- Ngày hoàn thành: 26/10/2026